

Partie 4 : Génération augmentée par récupération (RAG)

Bassem Ben Hamed

ENETCOM – Université de Sfax

Février 2026

Section 4.1 — Fondements du RAG

- ❶ Pourquoi les LLMs seuls ne suffisent pas
- ❷ Définition et architecture du RAG
- ❸ Retriever vs générateur : responsabilités
- ❹ Retrieval sparse, dense et hybride
- ❺ Embeddings, bases vectorielles et chunking
- ❻ Prompting et attribution des sources

Section 4.2 — RAG avancé et production

- ❼ Reranking et transformation de requêtes
- ❽ Retrieval multi-hop
- ❾ GraphRAG
- ❿ RAG agentique et garde-fous
- ⓫ Cadre d'évaluation et métriques
- ⓬ Architecture de production et Lab 6

À la fin de cette session, vous serez capable de :

- ❶ **Expliquer** le pipeline RAG complet, de l'ingestion à la génération ancrée.
- ❷ **Concevoir** des pipelines de retrieval robustes avec chunking, recherche hybride et reranking.
- ❸ **Décider** quand utiliser GraphRAG et comment l'implémenter de manière pragmatique.
- ❹ **Évaluer** la qualité du RAG avec des métriques par composant et de bout en bout.
- ❺ **Construire** des systèmes RAG orientés production avec observabilité et contrôle des coûts.
- ❻ **Comparer** les stratégies RAG, fine-tuning et hybride pour les cas d'usage entreprise.

Section 4.1

Fondements de la Génération Augmentée par Récupération

De la connaissance paramétrique aux réponses ancrées

Pourquoi les LLMs seuls ne suffisent pas en production

Limitations fondamentales

- Date de coupure et informations obsolètes
- Hallucinations en cas d'incertitude
- Pas d'accès direct aux données privées de l'entreprise
- Faible traçabilité des affirmations factuelles

Conséquences opérationnelles

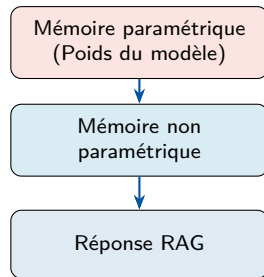
- Risques de conformité et d'audit
- Dégradation de la confiance utilisateur
- Coût élevé du fine-tuning fréquent
- Adaptation lente aux connaissances évolutives

Qu'est-ce que la génération augmentée par récupération ?

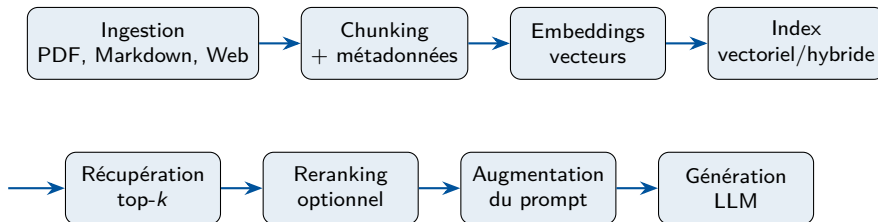
Définition

Le RAG combine une couche de recherche d'information avec un modèle de langage afin que les réponses soient générées à partir de **preuves récupérées** plutôt que de la seule mémoire paramétrique.

- Flux canonique : **Récupérer** → **Augmenter** → **Générer**
- Séparation de la mémoire :
 - **Paramétrique** : poids du modèle
 - **Non paramétrique** : documents, index vectoriel, graphe de connaissances
- Objectif : améliorer la factualité, la fraîcheur et



Workflow RAG de bout en bout



Principe d'ingénierie clé

La qualité de la réponse est généralement bornée par la qualité de la récupération. Une meilleure génération ne peut pas compenser entièrement de mauvaises preuves.

Retriever vs générateur : responsabilités

Retriever

- Maximiser le rappel des preuves
- Retourner des chunks pertinents, divers et récents
- Appliquer les filtres de métadonnées (tenant, date, domaine)
- Fournir des candidats pour le reranking

Générateur

- Synthétiser la réponse à partir du contexte récupéré
- Refuser les affirmations non étayées
- Citer les sources explicitement
- Garder un raisonnement cohérent et concis

Note pratique : les pipelines modulaires sont préférés en production pour le débogage et l'observabilité.

Retrieval sparse vs dense vs hybride

Approche	Forces	Faiblesses	Cas d'usage
Sparse (BM25)	Correspondance exacte de mots-clés, rapide	Manque les paraphrases sémantiques	Clauses juridiques, codes produits, logs
Dense (Embeddings)	Correspondance sémantique, robuste aux reformulations	Peut manquer des tokens rares exacts	Q&R en langage naturel, recherche conceptuelle
Hybride (BM25 + Vecteur)	Meilleur rappel et robustesse	Plus de complexité de réglage	Assistants entreprise, corpus mixtes

Les méthodes de fusion comme RRF apportent souvent des améliorations significatives avec une complexité limitée.

Prompting pour une génération ancrée

Politique de prompt

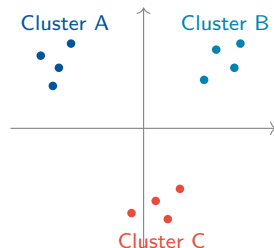
- Répondre uniquement à partir du contexte fourni
- Si la preuve manque : indiquer « preuve insuffisante »
- Joindre une citation pour chaque affirmation factuelle
- Séparer les faits de l'inférence si nécessaire

Format de citation

- [doc_id:chunk_id]
- [policy-v2:117]
- [handbook-rh:42]

Les embeddings en pratique

- Les embeddings encodent la proximité sémantique, pas la vérité factuelle.
- La performance dépend du domaine (biomédical, juridique, finance, code).
- La qualité multilingue varie fortement selon les modèles.



Axes de choix du modèle

- Qualité de retrieval sur votre benchmark (Recall@k, nDCG)
- Contraintes de latence et de débit
- Coût par million de tokens
- Résidence des données et contraintes de gouvernance

Pourquoi l'indexation ANN ?

- Recherche rapide de plus proches voisins à grande échelle
- Compromis latence/qualité contrôlable
- Valeur par défaut : index de la famille HNSW

Capacités de la plateforme

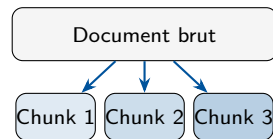
- Filtrage par métadonnées
- Recherche hybride
- Upsert et réindexation incrémentale
- Isolation multi-tenant

Chunking : le levier qualité numéro 1

- **Trop petit** : contexte fragmenté, réponse incomplète
- **Trop grand** : retrieval bruité, surcoût en tokens
- **Cible** : chunks sémantiquement cohérents et autonomes

Stratégies courantes

- Fenêtres de tokens de taille fixe
- Découpage récursif avec recouvrement
- Chunking sémantique basé sur les frontières
- Chunking adapté au format (Markdown, PDF, code)



Règle d'or

Ajuster la taille des chunks et le recouvrement avec des métriques de retrieval, pas l'intuition.

Modes d'échec typiques du RAG

- ❶ **Échec de récupération** : la preuve pertinente n'est pas récupérée.
- ❷ **Pollution du contexte** : les chunks récupérés contiennent du contenu non lié.
- ❸ **Inversion de classement** : le meilleur chunk n'est pas dans les premières positions.
- ❹ **Mauvaise utilisation du prompt** : le contexte est présent mais ignoré par le générateur.
- ❺ **Synthèse non étayée** : la réponse va au-delà des preuves.

Ordre de débogage

Récupération → Reranking → Prompting → Génération.

- Le RAG améliore la factualité en ancrant la génération dans des preuves externes.
- Des systèmes robustes nécessitent un bon chunking, une stratégie de retrieval et des citations.
- Le retrieval dense seul est rarement suffisant pour les corpus entreprise.
- Les métriques doivent guider chaque choix de conception.

Suite : techniques avancées pour les questions difficiles et les contraintes de production

Section 4.2

Techniques RAG avancées

Retrieval hybride, GraphRAG, RAG agentique et évaluation

Pattern du pipeline

- Exécuter BM25 et recherche vectorielle en parallèle
- Fusionner les candidats
- Appliquer la fusion de rangs (RRF ou pondérée)
- Transmettre les meilleurs candidats au reranker

Pourquoi ça marche

- Capture les termes exacts et la sémantique
- Réduit les angles morts de chaque méthode
- Améliore généralement le Recall@k

Reranking pour la précision au top- k

- L'étape de retrieval optimise la couverture (rappel).
- L'étape de reranking optimise l'ordonnancement (précision).

Rerankers Cross-Encoder

- Haute qualité de classement
- Prédiction conjointe de la pertinence (requête + chunk)
- Coût de latence accru

Reranking par LLM

- Flexible et explicable
- Coût de calcul plus élevé
- Utile pour les requêtes de domaines complexes

- **Reformulation** : lever l'ambiguïté
- **Expansion** : injecter des synonymes du domaine
- **Multi-requête** : diversifier les angles de recherche
- **HyDE** : utiliser un document hypothétique généré pour la recherche

Bénéfice attendu

Meilleure couverture des preuves pour les questions sous-spécifiées ou complexes.

Problème

Certaines questions nécessitent de relier des preuves provenant de plusieurs documents.

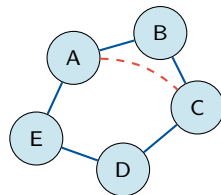
- Décomposer la question utilisateur en sous-questions.
- Récupérer les preuves de manière itérative pour chaque sous-étape.
- Agréger les résultats et vérifier la cohérence avant la réponse finale.
- Ajouter des vérifications explicites de confiance et de citations.

Ce pattern est souvent obligatoire pour les charges analytiques et de conformité.

GraphRAG : pourquoi et quand ?

Limite du retrieval par chunks

Le retrieval par chunks peut manquer les relations explicites entre entités distribuées dans différents documents.



Les liens entre entités permettent le raisonnement multi-documents

Pipeline de construction GraphRAG

- 1 Extraire les entités et relations depuis les documents.
- 2 Canonicaliser les noms d'entités (alias, doublons, normalisation).
- 3 Construire un graphe typé (nœuds, arêtes, timestamps, métadonnées).
- 4 Détecter les communautés/sous-graphes et créer des résumés locaux.
- 5 Persister les artefacts du graphe pour la traversée au moment de la récupération.

Contrainte clé

La qualité du graphe dépend directement de la précision de l'extraction et de la conception du schéma.

Stratégies de retrieval GraphRAG

Retrieval local

- Partir des entités ancrées dans la requête
- Traverser les nœuds voisins et les chemins de relations
- Récupérer les chaînes de preuves à fort signal

Retrieval global

- Router à travers les communautés du graphe
- Construire des résumés du global vers le local
- Fusionner les preuves du graphe avec le retrieval vectoriel

Limites et compromis du GraphRAG

- Coût d'ingénierie plus élevé que le RAG vectoriel standard.
- Maintenance continue nécessaire à mesure que les corpus évoluent.
- Risque de propagation d'erreurs depuis une extraction incorrecte des entités.
- Complexité opérationnelle accrue (stockage du graphe, mises à jour, gouvernance).

Règle de décision

Utiliser GraphRAG quand la structure relationnelle est centrale aux questions utilisateur, pas par défaut.

- L'agent décide **si**, **quand** et **comment** la récupération doit se faire.
- Routage dynamique de stratégie : vectoriel seul, hybride ou graphe en priorité.
- Boucles de réflexion pour vérifier la suffisance des preuves avant de répondre.
- Utilisation d'outils multi-étapes pour les questions difficiles.

Variantes avancées

Self-RAG, Corrective RAG (CRAG) et Adaptive RAG.

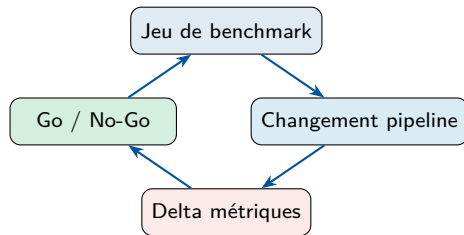
Contrôles de fiabilité

- Détection de preuves contradictoires
- Vérification obligatoire des citations
- Politique de repli en cas de preuves insuffisantes

Contrôles de sécurité

- Filtrage d'injection de prompt via les documents
- Assainissement du contenu avant le prompting
- Application des politiques d'accès aux données

- **Évaluation par composant** : ingestion, récupération, reranking, génération.
- **Évaluation de bout en bout** : de la question utilisateur à la qualité de la réponse.
- Construire des jeux de benchmarks représentatifs du domaine (golden set).
- Combiner métriques automatisées et revue humaine experte.



Principe

Pas de mise en production sans vérification de régression par rapport aux métriques de référence.

Métriques clés des systèmes RAG

Couche	Métriques	Interprétation
Récupération	Recall@k, Hit Rate, nDCG, MRR	Les preuves pertinentes sont-elles trouvées et bien classées ?
Génération	Faithfulness, Answer Relevancy	La réponse est-elle étayée et utile ?
Utilisation du contexte	Context Precision, Context Recall	Les chunks récupérés sont-ils utilisés et suffisants ?
Ops système	Latence p50/p95, coût/requête, taux de repli	Le système est-il fiable et économiquement viable ?

Exemples d'outils : Ragas, ARES et pipelines d'évaluation internes.

Protocole d'évaluation recommandé

- 1 Définir les tranches de cas d'usage (FAQ, politique, analytique, multi-hop).
- 2 Construire des questions de référence avec des réponses de confiance.
- 3 Établir la baseline des métriques du pipeline actuel.
- 4 Changer un seul levier à la fois (chunking, modèle, reranker, prompt).
- 5 Suivre les régressions et appliquer des seuils go/no-go.

Anti-pattern

Se fier uniquement à une démo manuelle sans évidence quantitative.

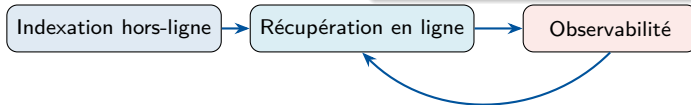
Architecture RAG de production

Plan hors-ligne

- Connecteurs et ingestion
- Nettoyage, déduplication, versioning
- Jobs d'embeddings et d'indexation
- Rafraîchissement planifié et événementiel

Plan en ligne

- Réécriture de requêtes et récupération
- Reranking et génération
- Cache (requêtes, embeddings, réponses)
- Limites de débit et contrôle du budget tokens



Essentiels d'observabilité

- Traces par requête (chunks récupérés + scores)
- Tableaux de bord latence et coûts
- Monitoring de conformité des citations
- Alertes de dérive sur les métriques clés

Anti-patterns courants

- Un seul index pour tous les domaines métier
- Chunking uniforme pour tous les types de documents
- Ignorer les contraintes de fraîcheur des documents
- Pas de pipeline d'évaluation continue

Section 4.3

Lab 6 : Construction de systèmes RAG de production

Du proof of concept à la qualité mesurable

Outils

LangChain, LlamaIndex, Chroma/Weaviate, Ragas.

- Ingérer et normaliser des données multi-format (PDF, Markdown, Web).
- Comparer les stratégies de chunking et mesurer leur impact.
- Implémenter un retrieval hybride avec reranking.
- Ajouter l'attribution de sources et les filtres de métadonnées.
- Construire un assistant « discutez avec vos données » avec comportement de repli.

Lab 6 – Parcours avancé (GraphRAG + Agentique)

- Construire un mini graphe de connaissances à partir du même corpus.
- Implémenter un retrieval hybride graphe + vecteurs.
- Ajouter un routeur de stratégie (vectoriel seul, hybride, graphe en priorité).
- Évaluer les gains et compromis :
 - qualité (faithfulness, pertinence)
 - latence et coût
- Documenter les cas d'échec et la stratégie d'atténuation.

RAG vs fine-tuning vs stratégie hybride

Critère		RAG	Fine-Tuning	Hybride
Connaissances fraîches		✓	✗	✓
Adaptation du comportement		Limité	✓	✓
Traçabilité des sources		✓	✗	✓
Vitesse d'itération		Élevée	Moyenne/Faible	Moyenne
Complexité ops		Moyenne	Élevée (entraînement)	Élevée

Configuration entreprise typique : fine-tuning pour le comportement + RAG pour les connaissances dynamiques.

Trois messages clés

- ❶ La qualité de la récupération détermine fortement la qualité de la réponse.
- ❷ Les gains majeurs viennent souvent du chunking, de la recherche hybride et du reranking.
- ❸ GraphRAG est puissant quand la structure relationnelle guide les questions utilisateur.

Suite : Partie 5

Fine-tuning et adaptation de modèles : quand et comment modifier le comportement du modèle lui-même.